

Natural Language Processing Basics

Turning human language into data that machines can analyze and generate

In this guide

- Introduction
- 1. Text Preprocessing
- 2. Tokenization, Stemming and Lemmatization
- 3. Representing Text as Numbers
- 4. Building a Text Classifier
- 5. Deep Learning, Transformers and Large Language Models
- 6. Evaluation, Ethics and Practical Advice
- Conclusion

Introduction

Natural Language Processing (NLP) is the field of artificial intelligence concerned with enabling computers to read, interpret and generate human language. Text and speech are among the richest sources of information in the world, and NLP is how software makes sense of them: filtering spam, translating languages, summarizing documents, answering questions and powering chat assistants.

Language is hard for machines. Words are ambiguous ('bank' of a river or a bank account), meaning depends on context, grammar has many exceptions, and people use slang, sarcasm and typos. NLP therefore combines linguistics, statistics and machine learning. This guide covers the core pipeline from raw text to modern transformer models.

Common NLP tasks

Task	Description	Example
Text classification	Assign a label to a text	Spam detection, topic tagging
Sentiment analysis	Detect opinion or emotion	Positive or negative product review
Named entity recognition	Find people, places, organizations	'Apple' the company in a news article
Machine translation	Convert between languages	English to Hindi
Summarization	Condense long text	News digest
Question answering	Answer from text or knowledge	Chatbots, search
Text generation	Produce new text	Drafting emails, code

What you will learn

- 1 How to clean and normalize raw text.
- 2 How to break text into tokens and reduce words to base forms.
- 3 How to convert text into numbers: bag of words, TF-IDF and embeddings.
- 4 How to build a simple classifier.
- 5 How transformers and large language models changed the field.
- 6 How to evaluate NLP systems and use them responsibly.

Tools: Examples use Python with scikit-learn, NLTK/spaCy and Hugging Face Transformers. Install with `pip install scikit-learn nltk spacy transformers`.

1. Text Preprocessing

Raw text is messy. Preprocessing converts it into a cleaner, more consistent form so that models focus on meaning rather than formatting noise. The exact steps depend on the task, and modern deep-learning models need far less cleaning than classic methods.

Typical cleaning steps

- 1 **Lowercasing:** treat 'Apple' and 'apple' as the same word (be careful when case carries meaning, as in named entities).
- 2 **Removing noise:** HTML tags, URLs, extra whitespace, special characters.
- 3 **Handling punctuation and numbers:** remove, keep or replace with a placeholder such as `<NUM>`.
- 4 **Removing stop words:** very frequent words like 'the', 'is', 'and' that carry little topical information.
- 5 **Normalizing:** fix contractions ('don't' becomes 'do not'), spelling variants and accents.

```
import re

def clean(text):
    text = text.lower()
    text = re.sub(r"https?://\S+", " ", text)          # remove URLs
    text = re.sub(r"<[^>]+>", " ", text)              # remove HTML tags
    text = re.sub(r"^[a-z0-9\s']", " ", text)         # keep letters, digits, spaces
    return re.sub(r"\s+", " ", text).strip()

print(clean("Loved it!! Visit https://x.co <b>NOW</b> :"))
# loved it visit now
```

Do not over-clean: Removing the word 'not' as a stop word would flip the meaning of 'not good'. Sentiment tasks often keep stop words and punctuation such as '!'.

Key points

- Preprocessing should serve the task; test whether each step helps.
- Keep a reproducible function so training and production text are cleaned identically.
- Transformer models usually work best on lightly cleaned text.

2. Tokenization, Stemming and Lemmatization

Computers work with numbers, so text must first be split into units. **Tokenization** divides text into tokens, which may be words, sub-words or characters.

Word and sentence tokenization

```
import nltk
nltk.download("punkt", quiet=True)
from nltk.tokenize import word_tokenize, sent_tokenize

text = "NLP is fun. Dr. Rao's models can't sleep!"
print(sent_tokenize(text))      # two sentences
print(word_tokenize(text))
# ['NLP', 'is', 'fun', '.', 'Dr.', 'Rao', "'s", 'models', 'ca', "n't", 'sleep', '!']
```

Simple splitting on spaces fails on punctuation and contractions, so libraries use language-aware rules. Modern models use **sub-word tokenization** (Byte-Pair Encoding, WordPiece), which splits rare words into smaller pieces such as 'un', 'believ', 'able'. This handles unseen words gracefully and keeps the vocabulary compact.

Stemming versus lemmatization

Both reduce inflected words to a common base so that 'running', 'ran' and 'runs' can be treated similarly.

Technique	How it works	Example	Trade-off
Stemming	Chops suffixes with rules	studies -> studi	Fast but crude; may produce non-words
Lemmatization	Uses a dictionary and grammar	studies -> study, better -> good	More accurate but slower

```
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp("The children were running and studying better methods")
print([t.lemma_ for t in doc])
# ['the', 'child', 'be', 'run', 'and', 'study', 'well', 'method']
```

Part-of-speech tagging and named entities

Beyond tokens, spaCy can label each word's grammatical role and detect entities.

```
doc = nlp("Google opened a new office in Bengaluru in 2024.")
for ent in doc.ents:
    print(ent.text, ent.label_)      # Google ORG / Bengaluru GPE / 2024 DATE
```

Key points

- Tokenization is the first step of every NLP pipeline.

- Lemmatization is preferred over stemming when accuracy matters.
- Sub-word tokenization lets neural models handle any word.

3. Representing Text as Numbers

Machine learning models need numeric input, so we must convert text into vectors. Representation is the most important design choice in NLP, and the methods have evolved from counting words to learning meaning.

Bag of words

Bag of words counts how many times each vocabulary word appears in a document, ignoring order. Each document becomes a long sparse vector. It is simple and surprisingly effective, but it cannot tell 'dog bites man' from 'man bites dog'.

```
from sklearn.feature_extraction.text import CountVectorizer

docs = ["I love NLP", "I love machine learning", "NLP loves language"]
cv = CountVectorizer()
X = cv.fit_transform(docs)
print(cv.get_feature_names_out())
print(X.toarray())
```

TF-IDF

Raw counts overweight common words. **TF-IDF** (term frequency times inverse document frequency) boosts words that are frequent in one document but rare across the collection, so distinctive words receive higher weights.

- **TF**: how often the word appears in this document.
- **IDF**: log of (total documents / documents containing the word); rare words score higher.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(ngram_range=(1, 2), stop_words="english")
X = tfidf.fit_transform(docs)
print(X.shape)          # (3 documents, number of unigrams and bigrams)
```

`ngram_range=(1, 2)` adds two-word phrases such as 'machine learning', which recovers some word order.

Word embeddings

Embeddings represent each word as a dense vector of a few hundred numbers learned from large text collections, placing words with similar meanings close together. Methods such as Word2Vec and GloVe capture relationships: the vector for 'king' minus 'man' plus 'woman' lands near 'queen'. Similarity between vectors is usually measured with **cosine similarity**.

Method	Captures meaning?	Vector type	Best for
Bag of words	No	Sparse counts	Simple baselines
TF-IDF	Partly (word importance)	Sparse weights	Search, classification
Word2Vec / GloVe	Yes (static)	Dense	Similarity, features
Contextual (BERT)	Yes (depends on context)	Dense	State-of-the-art tasks

4. Building a Text Classifier

Text classification is the most common NLP application. The classic recipe combines a vectorizer with a simple model and evaluates on held-out data. Scikit-learn's **Pipeline** keeps both steps together so that the same transformations run in training and prediction.

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

texts = ["great movie, loved it", "terrible plot and bad acting",
         "wonderful performance", "boring and far too long",
         "an absolute delight", "worst film I have seen",
         "fantastic story", "dull and disappointing"] * 25
labels = [1, 0, 1, 0, 1, 0, 1, 0] * 25          # 1 = positive, 0 = negative

X_tr, X_te, y_tr, y_te = train_test_split(texts, labels, test_size=0.2,
                                          random_state=0, stratify=labels)

clf = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
    ("model", LogisticRegression(max_iter=1000)),
])
clf.fit(X_tr, y_tr)
print(classification_report(y_te, clf.predict(X_te)))
print(clf.predict(["what a fantastic delight", "dull, boring mess"]))
```

Toy data: This example repeats eight sentences so it runs instantly. Real projects need thousands of diverse, labeled examples, such as the IMDB review dataset.

Steps to build a real classifier

- 1 Collect and label representative data.
- 2 Split into training, validation and test sets.
- 3 Preprocess and vectorize text.
- 4 Train a baseline (Naive Bayes or logistic regression).
- 5 Evaluate with precision, recall and F1, not only accuracy.
- 6 Inspect errors and the most influential words; iterate.

Other useful classic models

- **Multinomial Naive Bayes:** fast, strong baseline for word counts.
- **Linear SVM:** often excellent with TF-IDF features.
- **Topic models (LDA):** discover themes in unlabeled documents.

Evaluation reminders

- Use a confusion matrix to see which classes are mixed up.
- For imbalanced classes, examine per-class recall and the F1 score.
- Check for data leakage, such as duplicate texts appearing in both train and test sets.

5. Deep Learning, Transformers and Large Language Models

Earlier neural approaches such as recurrent networks (RNNs, LSTMs) read text one word at a time and struggled with long sentences. The **transformer** architecture, introduced in 2017, replaced this with **self-attention**, a mechanism that lets every word directly look at every other word in the input and weigh how relevant each is. This made training highly parallel and very effective at capturing context.

Key ideas

- **Attention:** the model learns which other words matter when interpreting each word. In 'The animal didn't cross the street because it was tired', attention helps link 'it' to 'animal'.
- **Pretraining:** a model first learns general language patterns from huge amounts of text using self-supervised objectives.
- **Fine-tuning and transfer learning:** the pretrained model is then adapted to a specific task with a small labeled dataset.
- **Contextual embeddings:** the same word gets different vectors in different sentences.

Families of models

Type	Examples	Strength	Typical use
Encoder	BERT, RoBERTa	Understands text	Classification, NER, search
Decoder	GPT-style models, Claude, Llama	Generates text	Chat, writing, code
Encoder-decoder	T5, BART	Transforms text to text	Translation, summarization

Using pretrained models with Hugging Face

```
from transformers import pipeline

sentiment = pipeline("sentiment-analysis")
print(sentiment("The support team resolved my issue quickly!"))
# [{'label': 'POSITIVE', 'score': 0.99...}]

summarizer = pipeline("summarization")
ner = pipeline("ner", aggregation_strategy="simple")
print(ner("Sundar Pichai spoke in Mountain View."))
```

Large language models (LLMs)

LLMs are very large decoder transformers trained to predict the next token. At scale they display broad abilities: answering questions, writing, translating and reasoning. You interact with them through **prompts**, and can improve results by giving clear instructions, examples (few-shot prompting) and relevant context. **Retrieval-augmented generation (RAG)** supplies documents from your own knowledge base so answers are grounded in real sources.

Limitations: LLMs can produce fluent but incorrect statements ('hallucinations'), reflect biases in training data, and have limited context windows. Verify important outputs.

6. Evaluation, Ethics and Practical Advice

Building an NLP system is only half the job; you must also measure it and use it responsibly.

Evaluation metrics by task

Task	Common metrics
Classification / NER	Precision, recall, F1, accuracy
Machine translation	BLEU, chrF, human review
Summarization	ROUGE, human judgment of faithfulness
Language modeling	Perplexity
Question answering	Exact match, F1, human evaluation
Retrieval / search	Precision at k, recall at k, MRR

Automated scores are proxies. For generation tasks especially, sample outputs and review them manually, ideally with domain experts.

Ethical considerations

- **Bias:** models learn stereotypes from text; test performance across dialects, genders and groups.
- **Privacy:** text may contain personal data; anonymize and follow data-protection laws.
- **Misinformation and misuse:** generated text can mislead; consider safeguards and transparency.

- **Language coverage:** many languages have far less data, so quality varies widely.

Practical advice

- 1 Start with a simple baseline (TF-IDF plus logistic regression) before reaching for large models.
- 2 Try a pretrained model first; fine-tune only if needed.
- 3 Invest in data quality and consistent labeling guidelines.
- 4 Monitor production inputs because language and topics drift over time.
- 5 Keep a human in the loop for sensitive decisions.

Common pitfalls

- Data leakage from duplicate or near-duplicate texts across splits.
- Over-aggressive preprocessing that removes signal.
- Evaluating only on clean, in-domain text and being surprised by real-world messiness.
- Trusting a fluent generated answer without checking its facts.

Conclusion

NLP bridges human language and computation. You have followed the path from cleaning and tokenizing raw text, through numeric representations such as bag of words, TF-IDF and embeddings, to building and evaluating a classifier, and finally to transformers and large language models that now define the state of the art.

Key takeaways

- Preprocess with the task in mind; do not remove information the model needs.
- Text becomes useful once it is represented numerically; better representations capture more meaning.
- A TF-IDF and linear-model baseline is fast, strong and easy to interpret.
- Transformers use attention to model context and can be reused via pretraining and fine-tuning.
- Evaluate carefully and consider bias, privacy and reliability.

Suggested next steps

- 1 Complete a small project: sentiment analysis on product reviews or a spam filter.
- 2 Explore the Hugging Face course and fine-tune a small BERT model.
- 3 Learn about embeddings for semantic search and retrieval-augmented generation.
- 4 Read about prompt design and evaluation of large language models.

Remember: Language is messy and human. The best NLP work pairs strong models with careful data, honest evaluation and respect for the people the text comes from.